

Formulação do Controle em Nível de Posição para um Manipulador 2-DOF (Baseado em Cinemática Inversa Analítica)

Gemini

29 de outubro de 2025

1 Objetivo e Abordagem

O objetivo é controlar a posição do efetuador final (EE) de um manipulador 2-DOF para que ele alcance uma posição de referência $\mathbf{x}_d = \begin{bmatrix} x_d \\ y_d \end{bmatrix}$ no espaço cartesiano.

Ao contrário das abordagens de "taxa resolvida" (nível de velocidade) que usam a Jacobiana para calcular velocidades de junta ($\dot{\mathbf{q}}$) iterativamente, esta solução opera no **nível de posição**.

A estratégia consiste em duas etapas:

1. **Resolver a Cinemática Inversa (CI) Analiticamente:** Encontrar a solução exata $\mathbf{q}_d = \begin{bmatrix} q_{1d} \\ q_{2d} \end{bmatrix}$ que satisfaz a equação de cinemática direta:

$$\mathbf{x}_d = f(\mathbf{q}_d)$$

Esta solução é calculada *uma única vez* quando o alvo é definido.

2. **Gerar uma Trajetória (Interpolação):** Criar um caminho suave no espaço de juntas, $\mathbf{q}(t)$, que começa na posição atual $\mathbf{q}_{\text{atual}}$ e termina na posição alvo $\mathbf{q}_d$ ao longo de uma duração T .

2 Etapa 1: Solução da Cinemática Inversa Analítica

Para um manipulador planar 2-DOF, podemos encontrar uma solução de forma fechada (analítica) usando geometria. A função `inverse_kinematics(x, y)` implementa esta solução.

2.1 Verificação de Alcance

Primeiro, verificamos se a posição alvo (x_d, y_d) é alcançável. A distância da base ao alvo, $D = \sqrt{x_d^2 + y_d^2}$, deve estar dentro dos limites do robô:

$$|L_1 - L_2| \leq D \leq L_1 + L_2$$

Onde L_1 e L_2 são os comprimentos dos elos. Se D estiver fora desse intervalo, a função retorna `None`, pois não existe solução.

2.2 Cálculo de q_2 (Ângulo do Cotovelo)

Considerando o triângulo formado pela base, o cotovelo e o efetuador final (com lados L_1 , L_2 e D), podemos usar a Lei dos Cossenos para encontrar o ângulo q_2 :

$$D^2 = L_1^2 + L_2^2 - 2L_1L_2 \cos(\pi - |q_2|)$$

Usando a identidade $\cos(\pi - \theta) = -\cos(\theta)$, e definindo q_2 como o ângulo interno, temos:

$$\cos(|q_2|) = \frac{D^2 - L_1^2 - L_2^2}{2L_1L_2}$$

O código escolhe uma das duas soluções possíveis (cotovelo para cima ou para baixo). A linha `q2 = -math.acos(...)` seleciona a configuração "cotovelo para cima" (assumindo uma convenção de ângulo).

2.3 Cálculo de q_1 (Ângulo da Base)

O ângulo q_1 é composto por dois ângulos:

1. α : O ângulo direto da base até o alvo, dado por $\alpha = \text{atan2}(y_d, x_d)$.
2. β : O ângulo formado entre o elo L_1 e a linha D .

Usando a Lei dos Cossenos novamente no mesmo triângulo, podemos encontrar β :

$$\cos(\beta) = \frac{D^2 + L_1^2 - L_2^2}{2L_1D}$$

O ângulo final q_1 é então a soma ou subtração desses ângulos, dependendo da configuração do cotovelo. Para a solução "cotovelo para cima" escolhida (q_2 negativo), temos:

$$q_1 = \alpha + \beta = \text{atan2}(y_d, x_d) + \arccos\left(\frac{D^2 + L_1^2 - L_2^2}{2L_1D}\right)$$

O resultado desta função é o vetor de ângulos alvo: $\mathbf{q}_d = \begin{bmatrix} q_{1d} \\ q_{2d} \end{bmatrix}$.

3 Etapa 2: Geração de Trajetória por Interpolação

Uma vez que $\mathbf{q}_d$ é conhecido, não o aplicamos instantaneamente, pois isso faria o robô "teleportar". Em vez disso, geramos uma trajetória suave no espaço de juntas usando **Interpolação Linear**.

Definimos um tempo de movimento T (MOVE_DURATION). O estado de animação τ é normalizado de 0 a 1:

$$\tau(t) = \min\left(\frac{t_{\text{decorrido}}}{T}, 1.0\right)$$

A posição da junta interpolada $\mathbf{q}(t)$ é então calculada como:

$$\mathbf{q}(t) = \mathbf{q}_{\text{início}} + (\mathbf{q}_{\text{alvo}} - \mathbf{q}_{\text{início}}) \cdot \tau(t) \quad (1)$$

Isso corresponde às linhas de código: `t = trajectory_time / MOVE_DURATION` e `q_current = q_start + diff * t`.

3.1 Tratamento de "Wrap-around" Angular

Um problema crítico na interpolação de ângulos é o "wrap-around" de 2π . Se $\mathbf{q}_{\text{início}} = -170^\circ$ e $\mathbf{q}_{\text{alvo}} = +170^\circ$, uma interpolação linear ingênua (Equação 1) moveria o robô 340° (o caminho longo).

Para garantir o caminho mais curto, a diferença de ângulo $\Delta\mathbf{q} = \mathbf{q}_{\text{alvo}} - \mathbf{q}_{\text{início}}$ deve ser normalizada para o intervalo $[-\pi, +\pi]$. Isso é feito no código:

- `diff = q_target - q_start`
- `diff[i] = (diff[i] + np.pi) % (2 * np.pi) - np.pi`

Esta nova `diff` representa o caminho angular mais curto, que é então usado na interpolação.

4 Conclusão

Esta abordagem em nível de posição é computacionalmente muito eficiente, pois a solução de CI analítica é calculada apenas uma vez. Ela não sofre de instabilidades de singularidade da mesma forma que os métodos baseados em Jacobiana.

A desvantagem é que esta é uma forma de controle em **malha aberta**. O robô simplesmente "executa" uma trajetória pré-calculada e não é reativo a perturbações ou a alvos que se movem dinamicamente.